

Nx vs Melos: сравнение инструментов оркестрации flutter-монорепо

Содержание

1. Обзор инструментов	1
Nx	1
Melos	2
2. Сравнительная таблица	4
3. Когда что выбирать	5

1. Обзор инструментов

Nx

Экосистема и аудитория. Nx — система сборки и оркестратор задач для монорепо, разработанная в первую очередь под язык TypeScript и связанные с ним фреймворки (Angular, React, Next.js, Vue, Node.js, NestJS). Конфигурация описывается в формате JSON и выполняется средой Node.js; для работы Nx требует установленный Node.js и менеджер пакетов из экосистемы Node (`npm`, `pnpm` или `yarn` — функциональные аналоги `flutter pub` для JavaScript-пакетов).

Поддержка языков и стеков, не входящих в экосистему Node.js, реализуется через подключаемые плагины. Для Dart и Flutter официального плагина от Nx Inc. на момент написания документа не существует. Используется community-плагин `@nxrocks/nx-flutter`, который оборачивает вызовы `flutter` и `dart` CLI и регистрирует их как задачи Nx. По модели работы это похоже на запуск произвольных shell-команд из `melos.yaml`, но с дополнительным слоем над ними.

Модель работы. Nx строит две связанные структуры:

- *Project graph* — граф проектов (аналог графа пакетов в Melos workspace). Зависимости восстанавливаются по импортам в исходниках и по объявлениям в манифестах пакетов экосистемы Node.js. Для Dart-пакетов восстановление зависимостей средствами Nx не выполняется — связи описываются вручную либо берутся из плагина.
- *Task graph* — граф задач, в котором задача одного пакета может явно зависеть от задач другого. Зависимости описываются полем `dependsOn`, например `dependsOn: ["^build"]` означает «перед запуском задачи требуется задача `build` во всех пакетах-зависимостях». Прямого аналога в Melos нет: граф в Melos строится на уровне пакетов, а не отдельных задач, поэтому описать зависимость задачи `test` пакета А от задачи `build` пакета В средствами Melos невозможно.

Команда `nx affected` принимает diff (по умолчанию относительно базовой ветки) и через project graph определяет множество пакетов, на которые повлияли изменения; задачи запускаются только на этом множестве. Функционально близко к фильтру `melos exec --diff`, отличие — в способе восстановления затронутого множества: Nx опирается на граф импортов, выводимый из исходников и манифестов, и работает на уровне отдельных задач; Melos определяет изменившиеся пакеты по git-diff на уровне файлов и расширяет множество через граф пакетов флагами `--include-dependents` и `--include-dependencies` на уровне пакета целиком.

Кеширование задач. Nx ведёт локальный кеш результатов задач. Для каждой задачи вычисляется хеш на основе входных данных (исходники, переменные окружения, конфигурация инструментария, явно объявленные `namedInputs`). При совпадении хеша задача не выполняется повторно — Nx воспроизводит сохранённый stdout/stderr и выходные файлы. Прямого аналога в Melos нет: `melos run test` выполняет тесты на всех пакетах, прошедших фильтр, независимо от того, изменились ли входы.

Удалённый кеш (remote cache) и распределённое выполнение задач (distributed task execution) предоставляются через коммерческий сервис Nx Cloud. Существуют ограниченные варианты self-hosted remote cache — коммерческий пакет Nx Powerpack и community-адаптеры под объектные хранилища S3/GCS.

Версионирование и публикация. Команда `nx release` поддерживает две модели версионирования: fixed (единая версия для всего набора пакетов) и independent (каждый пакет версионизируется отдельно). Поддерживаются conventional commits, автоматическая генерация changelog, создание git-тегов и публикация в реестры пакетов экосистемы Node.js (`npm`). Публикация в `pub.dev` средствами `nx release` не поддерживается.

Кодогенерация и расширяемость.

- *Generators* — скаффолдинг проектов, библиотек и файлов. Прямого аналога в Melos нет; в Dart-инструментарии скаффолдинг чаще выполняется через `mason` или собственные шаблоны.
- *Executors* — типизированные обёртки над командами. По функции близки к скриптам в секции `scripts` файла `melos.yaml`, но с явной схемой параметров.
- *Migrations* — автоматические преобразования кода и конфигов при обновлении версии плагина или самого Nx. Отдельного аналога в Melos нет.

Lifecycle-хуки в стиле pre/post-скриптов в Nx реализуются через `dependsOn` в описании задачи, а не через отдельные именованные хуки.

CI и облако. Nx Cloud — управляемый сервис, предоставляющий remote cache, распределённое выполнение задач (Nx Agents), сбор метрик и обнаружение нестабильных тестов. Без Nx Cloud Nx работает локально и в CI, но распределённое выполнение задач — функция исключительно облачного сервиса.

Зрелость. Мейнтейнер — компания Nx Inc. (ранее Nrwl). Релизы выходят регулярно. Звёзд на GitHub — порядка 28 тысяч. Применяется преимущественно в монорепо на TypeScript/JavaScript.

Кривая входа. Высокая. Требуется освоить понятия project graph, task graph, executors, generators, named inputs, inferred tasks, а также экосистему Node.js (менеджер пакетов, `node_modules`, `package.json`). Для Flutter-разработчика без предшествующего опыта в Node.js входной порог выше, чем у Melos, на сопоставимый объём функциональности.

Melos

Экосистема. Только Dart и Flutter. Поддержки других языков и плагиновой системы для них не предусмотрено. Пакеты, не описываемые `pubspec.yaml`, в workspace не включаются.

Модель работы. Workspace описывается в корневом `pubspec.yaml` (синтаксис Dart 3.6+ workspaces) либо в `melos.yaml`. Базовые команды:

- `melos bootstrap` — резолв зависимостей через `dart pub get` / `flutter pub get` по всем пакетам с применением path-зависимостей внутри workspace.
- `melos exec` — запуск произвольной команды по множеству пакетов с поддержкой фильтров (`--diff`, `--scope`, `--include-dependents`, `--no-private`, `--dir-exists`, `--file-exists`).
- `melos run` — запуск именованного скрипта, объявленного в секции `scripts`.

Граф пакетов workspace строится автоматически на основе секций `dependencies` и `dev_dependencies` манифестов `pubspec.yaml`. Этот граф используется для упорядочивания и параллелизации выполнения команд: при `melos exec` и `melos run` пакеты по умолчанию обрабатываются в порядке топологической сортировки, степень параллелизма задаётся флагом `--concurrency`, а опции конфигурации скриптов (`exec.orderDependents` и аналогичные) позволяют принудительно соблюдать порядок зависимостей при выполнении команды.

Топологический порядок означает, что пакет обрабатывается только после того, как команда отработала на всех пакетах, от которых он зависит. Например, при цепочке `app → ui_kit → core` команда выполнится сначала на `core`, затем на `ui_kit`, затем на `app`. Это свойство порядка, а не алгоритма: у одного графа допустимо несколько корректных топологических порядков, и Melos выбирает один из них. На практике это требуется в сценариях, где зависимости должны быть готовы до запуска команды на зависимом пакете — например, при последовательной публикации в `pub.dev` или при кодогенерации, артефакты которой потребляются другими пакетами. При `--concurrency > 1` топологический порядок сохраняется как ограничение: пакеты без взаимных зависимостей могут выполняться одновременно, но зависимый пакет всё равно ожидает завершения своих зависимостей.

Принципиальное отличие от Nx — гранулярность узлов графа, а не наличие графа как такового:

- У Melos узлами графа являются пакеты; команда выполняется на пакете целиком как единая единица работы.
- У Nx узлами являются отдельные задачи (`build` пакета A, `test` пакета A, `build` пакета B и т.д.), а зависимости между задачами разных пакетов задаются явно через `dependsOn`.

Из task-level гранулярности Nx вытекают возможности, отсутствующие у Melos: кеширование и инвалидация на уровне отдельной задачи, а также параллельное выполнение разнотипных задач разных пакетов при отсутствии явной зависимости между ними.

Кеширование задач. Кеш результатов задач отсутствует. Хеширование входов и пропуск задач не выполняются. Сокращение объёма выполняемой работы достигается фильтром `--diff` и средствами CI (например, кеширование `~/pub-cache` в GitHub Actions).

Версионирование и публикация. `melos version` определяет тип инкремента (`patch/minor/major`) по conventional commits, генерирует changelog для каждого пакета и сводный workspace changelog, создаёт git-теги, обновляет version constraints у пакетов-зависимостей внутри workspace. Поддерживаются graduate (переход `prerelease` → `stable`), `prerelease` с пользовательским tag, ручное переопределение версии. `melos publish` публикует пакеты в `pub.dev`.

Скрипты и хуки. Lifecycle-скрипты в `melos.yaml` представляют собой shell-команды с поддержкой pre/post-хуков. Кодогенерация (`build_runner`) выполняется внутри пакетов средствами Dart-инструментария; Melos вызывает соответствующую команду, но не вмешивается в её работу.

CI и облако. Управляемого облачного сервиса не предоставляется. Remote cache и распределённое выполнение задач отсутствуют. Локальная параллелизация в пределах одной машины поддер-

живается самым Melos через флаг `--concurrency` с учётом графа пакетов; распределение нагрузки между несколькими runners достигается средствами CI-системы (например, матрицей в GitHub Actions).

Зрелость. Мейнтейнер — Invertase (та же компания, что поддерживает FlutterFire). Релизы выходят реже, чем у Nx. Звёзд на GitHub — порядка 1.5 тысяч. Применяется в большинстве крупных Flutter-монорепо: FlutterFire, Flame, ряд пакетов экосистемы Bloc.

Кривая входа. Низкая. Конфигурация уместается в один YAML-файл; ключевые понятия — workspace, scripts, filters, version, publish.

2. Сравнительная таблица

Измерение	Nx	Melos
First-class языки	TypeScript/JavaScript, Angular, React, Node.js	Dart, Flutter
Polyglot-репо	Поддерживается через плагины, качество зависит от плагина	Не поддерживается
Поддержка Flutter/Dart	Через community-плагин <code>@nxrocks/nx-flutter</code>	Нативно
Поддержка JS/TS	Нативно	Не поддерживается
Гранулярность графа зависимостей	Task-level: project graph + task graph с явными <code>dependsOn</code> между задачами	Package-level: граф пакетов из <code>pubspec.yaml</code> ; зависимостей между отдельными задачами разных пакетов нет
Affected-команды	<code>nx affected</code> по project graph	Фильтр <code>--diff</code> по git
Локальный кеш задач	Поддерживается, по хешу входов	Не поддерживается
Remote cache	Через Nx Cloud (платно) или ограниченные self-hosted-варианты	Не поддерживается
Распределённое выполнение задач	Через Nx Cloud Agents (платно)	Не поддерживается
Версионирование и публикация	<code>nx release: independent/fixed, conventional commits, changelog</code> , публикация в <code>npm</code>	<code>melos version</code> и <code>melos publish: conventional commits, changelog</code> , публикация в <code>pub.dev</code>
Расширяемость	Generators, executors, migrations, плагины	Scripts и hooks в YAML, плагиновой системы нет
Внешние зависимости среды	Node.js и менеджер пакетов экосистемы Node.js	Dart SDK / Flutter SDK
CI и облако	Nx Cloud (управляемый сервис): remote cache, метрики, агенты	Только CI-система пользователя
Зрелость	Активный мейнтейнер, частые релизы, ~28k звёзд	Стабильный мейнтейнер, более редкие релизы, ~1.5k звёзд

Измерение	Nx	Melos
Vendor lock-in	Средний: без Nx Cloud распределённое выполнение недоступно	Низкий: open-source, без облачных компонентов
Кривая входа	Высокая	Низкая

3. Когда что выбирать

Монорепо только из Dart/Flutter-пакетов. Применяется Melos. Поддержка Flutter в Nx обеспечивается community-плагином и требует установки Node.js и менеджера пакетов экосистемы Node.js, что для Dart-проекта является дополнительной зависимостью среды без эквивалентной функциональной отдачи. Версионирование с учётом семантики `pub.dev` (формат `version constraints`, `prerelease`-теги, публикация) реализовано в Melos нативно; в Nx эквивалентная функциональность для `pub.dev` отсутствует.

Polyglot-монорепо (Flutter совместно с TypeScript/JavaScript-кодом). Решение зависит от соотношения объёмов. Если основной объём кода относится к экосистеме Node.js, а Flutter присутствует как отдельный модуль, применение Nx с подключением Flutter через `@nxrocks/nx-flutter` оправдано. Если основной объём кода относится к Dart/Flutter, а компоненты на других языках единичны, рассматривается раздельное управление: Melos для Dart-части, отдельный инструментарий для остальных частей. Совмещение Nx и Melos в одном workspace технически возможно, но влечёт дублирование конфигурации, двух CLI и двух моделей управления зависимостями.

Команда до пяти разработчиков. При прочих равных Melos имеет более низкую стоимость владения. Функциональные преимущества Nx (кеш задач, `distributed execution`) проявляются на репо порядка 20 и более пакетов и при значительном объёме CI-времени.

Open-source библиотека с публикацией пакетов. Для пакетов `pub.dev` применяется Melos: поддерживаются `changelog`, `prerelease`, `graduate`, публикация. Для пакетов экосистемы Node.js применяются `nx release` или `changesets`.